TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA
Macroarea di Ingegneria
Dipartimento di Ingegneria Civile e Ingegneria Informatica

OS-Level and Lightweight Virtualization

Corso di Sistemi Distribuiti e Cloud Computing
A.A. 2025/26

Valeria Cardellini

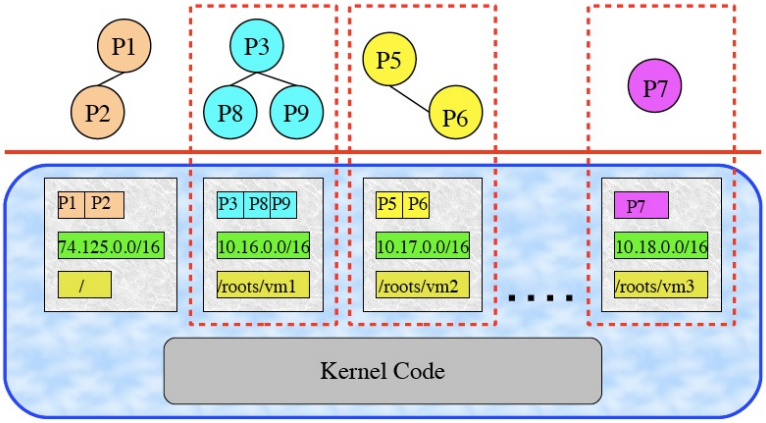
Laurea Magistrale in Ingegneria Informatica

OS-level virtualization

- Let's consider **operating system (OS) level virtualization** (or ***container-based virtualization***)
- It allows running multiple isolated (*sandboxed*) user-space instances on top of a single OS
  - Such instances are called:
    - **containers**
    - jails
    - zones

OS-level virtualization

- OS kernel allows the existence of multiple isolated user-space instances, called containers
- Each container has:
  - Its own set of processes, file systems, users, network interfaces with IP addresses, routing tables, firewall rules, …
- Containers share the same OS kernel (e.g., Linux)

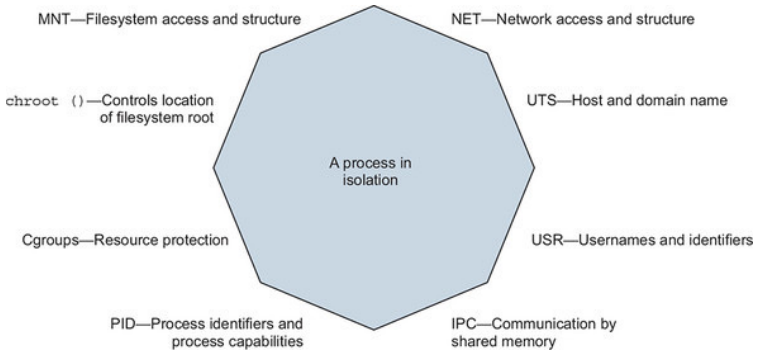


OS-level virtualization: mechanisms

- Which OS kernel mechanisms are used to manage containers?
  - Containers need to isolate processes from each other in terms of sw and hw (CPU, memory, …) resources
- Main mechanisms offered by Unix-like OS kernels
  - chroot (change root directory)
    - Allows changing the apparent root folder for the current running process and its children
  - cgroups (Linux-specific)
    - Manage resources for groups of processes, such as CPU and memory allocation
  - namespaces (Linux-specific)
    - Per-process resource isolation, ensuring that each container has its own isolated environment

Mechanisms: namespaces

- Feature of Linux kernel that allows to isolate what a set of processes can see in the operating environment
  - Includes resources such as processes, ports, files, …
- Kernel partitions resources so that one set of processes can see a specific set of resources, while another set of processes sees a different set of resources
- 6 different types of namespaces



Mechanisms: namespaces

- mnt: isolates mount points seen by a container
  - Virtually partitions the file system, so processes running in separate mount namespaces cannot access files outside of their mount point
- pid: isolates PID space, ensuring that each process only sees itself and its children (PID 1, 2, 3, …)
- network: allows each container to have its dedicated network stack
  - Its own private routing table, set of IP addresses, socket listings, firewall rules, and other network-related resources
- user: isolates user and group IDs
  - E.g., allows a non-root user on host to be mapped to root user within the container, without granting actual root access to host

Mechanisms: namespaces

- uts (Unix timesharing): provides dedicated host and domain names
  - Allows processes to think they are running on servers with different names, even though they share the same host
- ipc: provides dedicated shared memory for IPC
  - E.g., separate Posix message queues for different containers

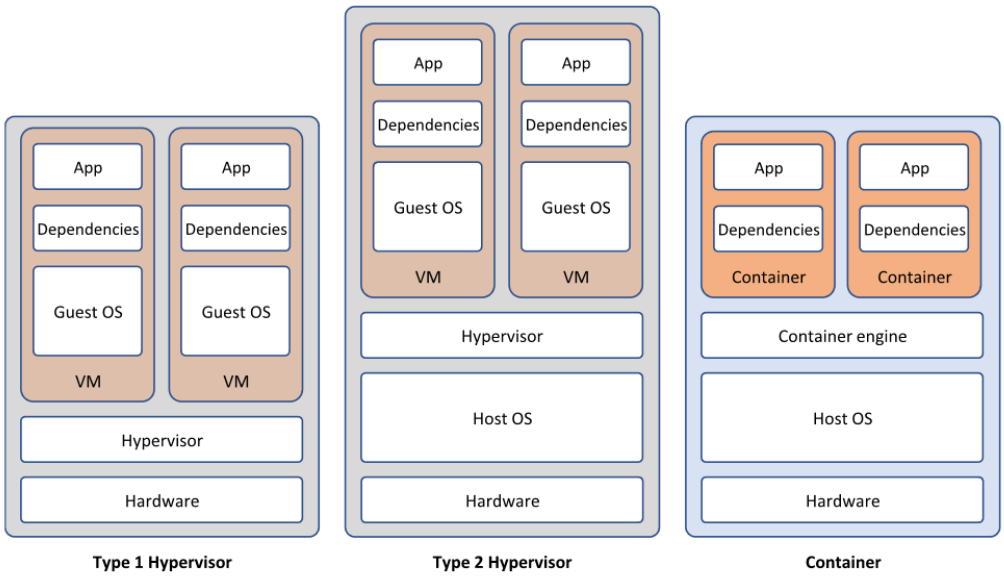
Mechanisms: cgroups

- cgroups = control groups
- Limit, measure and isolate the use of hw resources (CPU, memory, I/O, network) for a group of processes
- Exposed via a filesystem interface (similar to `sysfs` and `procfs`)
  - Default mount point: `/sys/fs/cgroup/`

- In a nutshell:
  - namespaces implement *information isolation*: what a container can see
  - cgroups implement *resource isolation*: how many resources a container can use

OS-level virtualization: pros

- VMM-based vs container-based virtualization



In a nutshell: lightweight vs. heavyweight

OS-level virtualization: pros

vs. VMM-based virtualization (type-1)

✓ Near-native performance
  – No VMM indirection for system calls

✓ Fast startup and shutdown
  – Seconds (even msec) per container vs. minutes per VM

✓ High density
  – Hundreds of containers per physical machine (PM)

✓ Small footprint
  – Container images are smaller since they exclude the OS kernel

✓ Memory efficiency
  – Containers can share memory pages on the same PM

✓ Portability and interoperability
  – Apps run across environments

OS-level virtualization: cons

vs. VMM-based virtualization (type-1)

✗ Less flexible
- Only supports native apps for the OS kernel (e.g., no Windows container on Linux host)
- Cannot run different OS kernels on same PM; however, can run multiple Linux distributions (e.g., Ubuntu, CentOS)

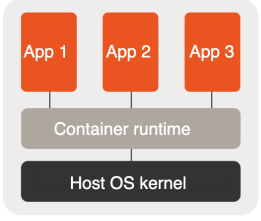
✗ Weaker isolation
- Process-level isolation leads to higher performance interference on shared resources

✗ Higher security risks
- A kernel vulnerability affects the entire system
- Since containers share the kernel, one compromised container can impact other containers and the host

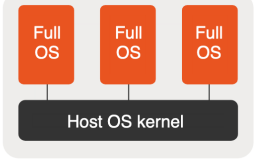
✗ Reduced hardware/device isolation
- Device passthrough is more difficult and less secure

OS-level virtualization: products

- Docker
  - Most popular container engine
  - Provides *application containers*
    - Package and run *a single application* with its dependencies
  - Supports Open Container Initiative (OCI) standards https://opencontainers.org
- LXC (LinuX Containers) https://linuxcontainers.org/lxc/
  - Supported by mainline Linux kernel
  - Provides *system containers*
    - Run a *whole OS user space with multiple processes*, but share the host kernel



Application containers



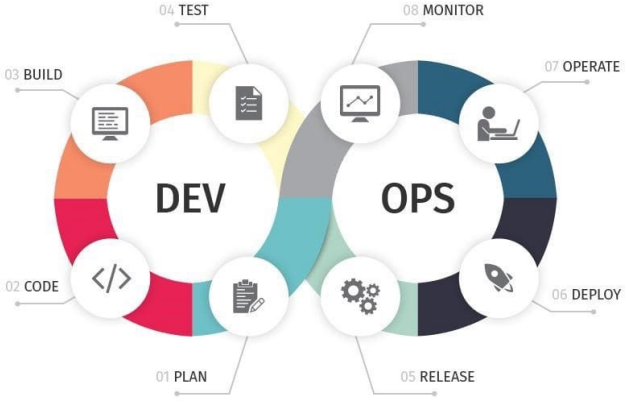
System containers

OS-level virtualization: products

- Podman https://podman.io
  - Supports OCI; Docker compatible CLI
  - Daemonless, rootless operation for improved security
- FreeBSD Jail
  - Strong process and filesystem isolation
- OpenVZ / Virtuozzo https://openvz.org
  - For Linux; primarily for system containers
- Non-Linux platforms
  - Windows and macOS support containers (e.g., Docker Desktop)
- Alternative approach
  - Install a Linux VM as a guest OS
  - Run container engines (e.g., Docker) inside the VM
  - Performance Impact: nested virtualization leads to reduced performance

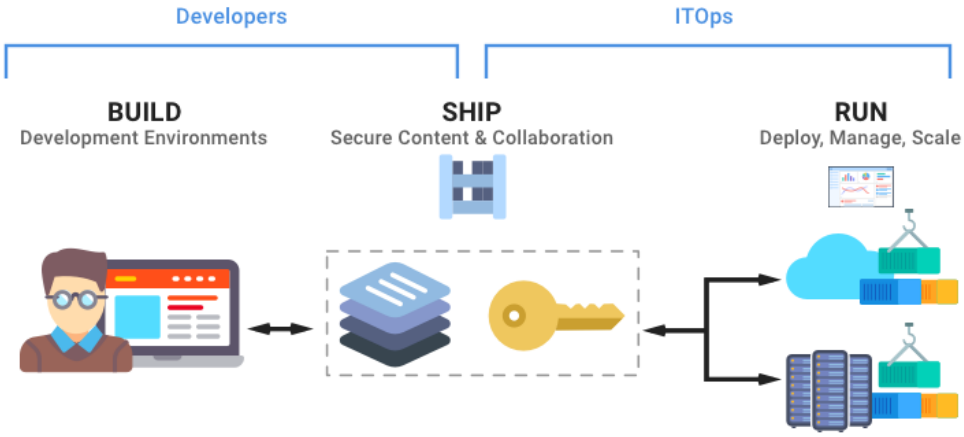
Containers, DevOps and CI/CD

- DevOps: development methodology that bridges the gap between Development and Operations, focusing on collaboration, continuous integration, and automated delivery
- CI/CD:
  - Continuous Integration (CI): merges developers' work into a shared codebase
  - Continuous Delivery (CD): ensures frequent and reliable releases



Containers, DevOps and CI/CD

- Containers: simplify building, packaging, sharing, and deploying apps with all dependencies
- Enable collaboration by sharing images, and streamline deployment across environments without extra configuration

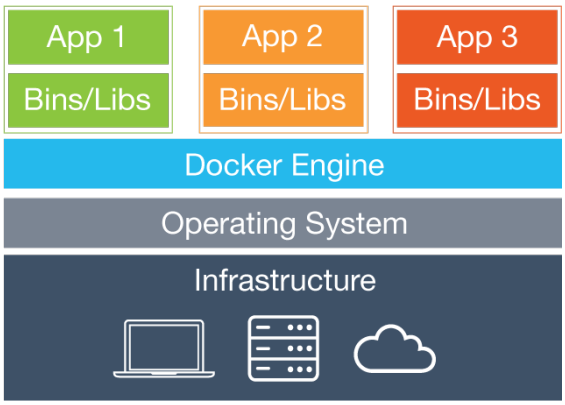


Containers, microservices, and serverless

- Using containers
  - Package apps and all dependencies into a single unit that runs almost anywhere
  - Use fewer resources than traditional VMs.
- Containers enable
  - Microservices: break down apps into small, independent services
  - Serverless: package functions into containers for efficient execution

Case study: Docker

- Lightweight and secure container-based virtualization
  - Application containers: contain the application and its dependencies, but share OS kernel with other containers
  - Isolation: containers run as isolated processes in user space on host OS
  - Portability: containers are infrastructure-agnostic and can run anywhere



Docker: features

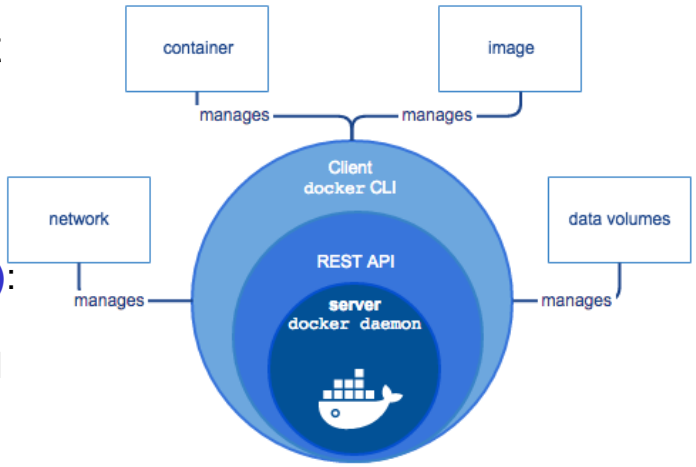
- Portable deployment
  - Easily deploy across different machines and environments
- Versioning
  - Git-like version control for container images
- Component reuse
  - Reuse components (e.g., libraries, services) via Docker images
- Shared libraries
  - Access to pre-built images on Docker Hub https://hub.docker.com
- OCI support
- Scalability
  - Works seamlessly with Kubernetes for scaling applications

Docker: internals

- Written in Go
- Exploits Linux kernel mechanisms for resource management and isolation (cgroups and namespaces)
  - Early versions were based on Linux Containers
  - Then transitioned to libcontainer, a Go-based container runtime: provides tools for managing containers with namespaces, cgroups, capabilities, and filesystem access controls https://pkg.go.dev/github.com/opencontainers/runc/libcontainer
  - runc: libcontainer is now part of runc, the CLI tool for spawning and run nning containers based on OCI specification https://github.com/opencontainers/runc

Docker Engine: architecture

- Docker Engine: core component of Docker that enables containerization
- Client-server application composed by
  - **Docker daemon (dockerd)**: server component that listens for API requests and manages Docker objects like images, containers, networks, and volumes

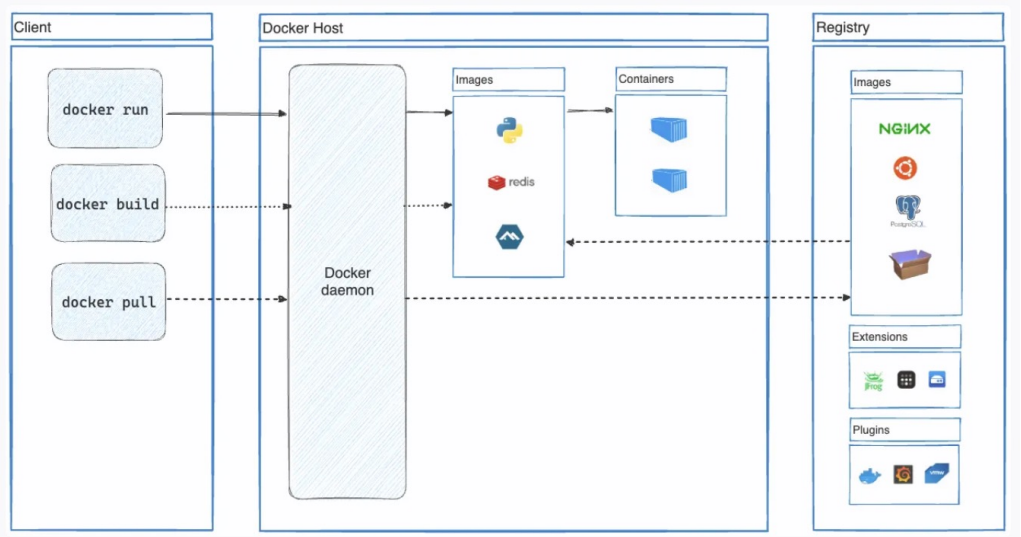


  - **REST API**: specifies the interfaces that programs can use to interact with Docker daemon for operations like creating, managing, and querying Docker objects
  - **CLI client**: allows users to send commands to Docker daemon via REST API

https://docs.docker.com/get-started/docker-overview/#docker-architecture

Docker: client-server architecture

- Docker client
  - Interface through which users interact with Docker
  - Sends commands to Docker Daemon to build, run, and distribute Docker containers
  - Client and daemon communicate through sockets or the REST API



Docker: images

- Read-only template
  - Used to create Docker containers, containing everything needed to run app (code, dependencies, configurations)
- Build component
  - Docker images enable distribution of apps with their runtime environments, removing the need to manually install packages
  - Target machine must be Docker-enabled
- **Dockerfile**
  - Text file with simple instructions that Docker uses to build images automatically
- **Image registry**
  - Images can be pulled and pushed to/from public or private registries
- Image naming
  - Format: `[registry/][user/]name[:tag]`
  - Default tag is latest

Docker image: Dockerfile

- Docker images are created from a Dockerfile and a context
  - Dockerfile: text file containing instructions to assemble the image
  - Context: set of files (e.g., app code, libraries) used during the image build process
  - Images often build on parent images (e.g., Alpine, Ubuntu)
- Dockerfile syntax

  `# Comment` (for comments)

  `INSTRUCTION arguments` (commands like RUN, COPY, etc.)
- Instructions in the Dockerfile run sequentially

Docker image: Dockerfile

- Common Dockerfile instructions
  - **`FROM <image>`**: specifies the parent image (mandatory unless you want to start from **`scratch`**)
  - **`WORKDIR <path>`**: sets the working directory inside the container
  - **`COPY <host-path> <image-path>`**: copies files from host to container image
  - **`RUN <command>`**: executes the specified command during image build
  - **`ENV <name> <value>`**: sets an environment variable
  - **`EXPOSE <port>`**: exposes a network port for the container
  - **`CMD ["<command>", "<arg1>"]`**: defines the default command to run when the container starts

https://docs.docker.com/get-started/docker-concepts/building-images/writing-a-dockerfile/

Docker image: Dockerfile

- Example: Dockerfile to build the image of a container that will run as application a simple todo list manager written in Node.js

```
# syntax=docker/dockerfile:1

FROM node:lts-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Directory with app code

```
├── getting-started-app/
│   ├── package.json
│   ├── README.md
│   ├── spec/
│   ├── src/
│   └── yarn.lock
```

https://docs.docker.com/get-started/workshop/02_our_app/

Docker image: build

- Building Docker image from Dockerfile and context
  - Context: set of files located in the specified PATH or URL
- Build command

```
$ docker build [OPTIONS] PATH | URL | -
```

- Example: to build an image for Node.js app (slide 24)

```
$ docker build -t getting-started .
```

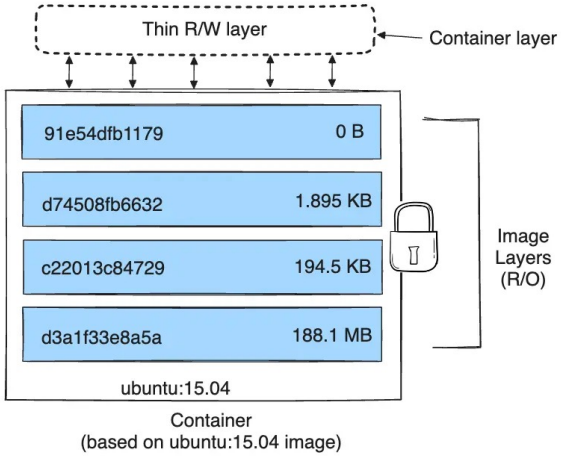
  - The `-t` flag is used to tag the image with a name (and optionally a version)
  - If Dockerfile is named something different from Dockerfile, use the `-f` flag:

```
$ docker build -t getting-started -f myDockerfile .
```

https://docs.docker.com/reference/cli/docker/build-legacy/

Docker image: layers

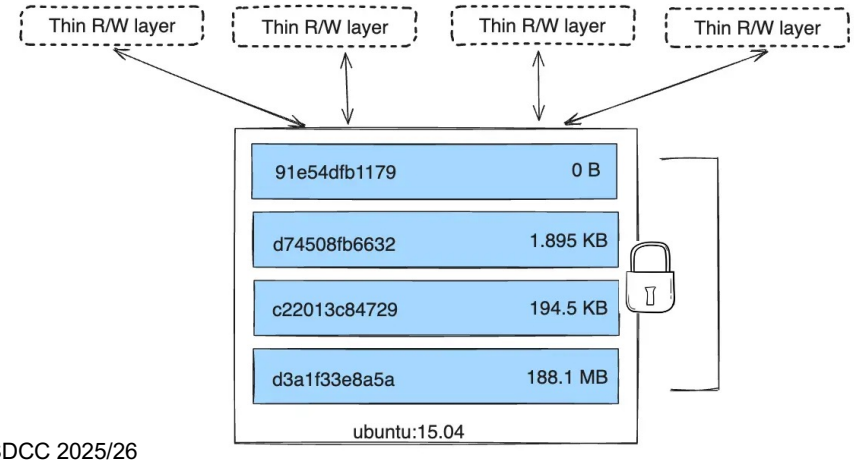
- Each Docker image consists of a series of layers
- Docker uses *union file systems* to combine these layers into a single unified view
  - These layers are stacked to form a base the base of a container's root file system
  - Docker leverages *copy-on-write* (CoW) strategy to optimize performance



Docker image: layers

- Layering pros
  - ✓ Efficient layer sharing and reuse: common layers are installed only once, saving bandwidth and storage space
  - ✓ Separation of concerns: allows better management of dependencies
  - ✓ Facilitates software specialization

https://docs.docker.com/storage/storagedriver



Docker image: layers and Dockerfile

- Instructions in the Dockerfile that modify the filesystem create a new layer
  - `FROM`, `COPY`, `RUN`, … add new layers
  - Instructions that modify the image's metadata (e.g., `CMD`, `LABEL`) do not create new layers
- All layers, except the last one, are read-only
- When a container is created from an image, a writable layer (aka *container layer*) is added on top
  - Changes made to a running container (e.g., writing a file) are written to the writable layer
  - These changes are ephemeral: do not persist once the container is deleted
  - Suitable for storing *ephemeral data* generated at runtime
- To inspect an image and view its metadata, including image layers: `$ docker inspect <imageid>`

Docker container storage

- Containers are usually stateless
- Why? Easier to:
  - Scale: start quickly new replicas
  - Restart: from failure
  - Migrate: move between hosts
- Very little data is written to container's writable layer
- Data is typically stored in **Docker volumes**
- However, some workloads require writing data to container's writable layer

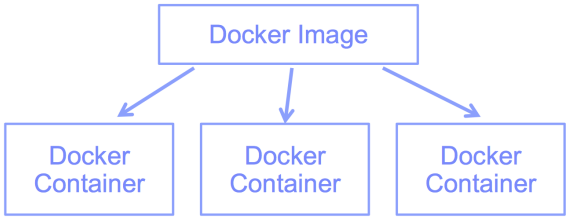
Docker: storage drivers

- Storage driver controls how *images* and *containers* are stored and managed on the host
- Popular storage drivers include:
  - Overlay2: file level, preferred for all Linux distros
  - Device Mapper
  - Btrfs
  - Zfs: block level
- Choosing the storage driver's affects container performance
  - Storage drivers are optimized for space efficiency, but write speeds can be lower than native file system performance

https://docs.docker.com/storage/storagedriver/select-storage-driver

Docker: containers and registry

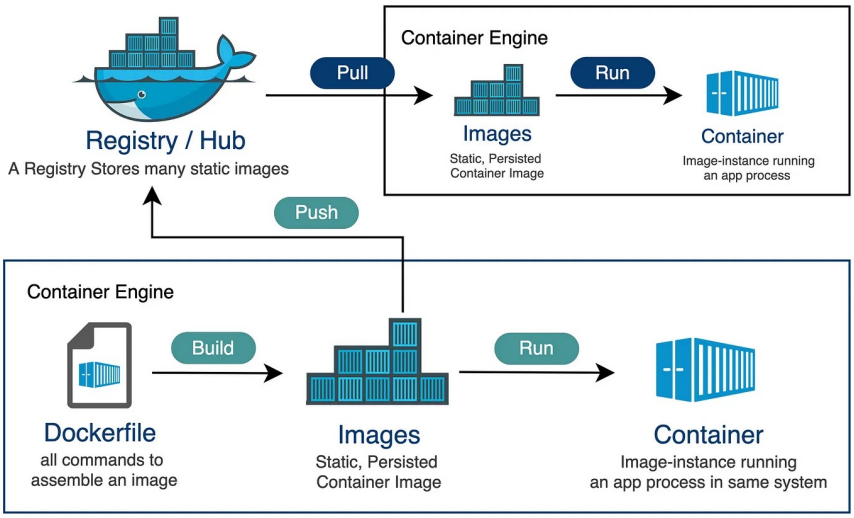
- Docker **container**: runnable instance of a Docker image
  - Containers are the ***run*** component of Docker
  - Run, start, stop, move, or delete a container using Docker API or CLI commands
  - Stateless nature of containers: when a container is deleted, any data written outside of data volumes is lost



- Docker **registry**: stateless server-side application that stores and distributes Docker images
  - Registry is the ***distribute*** component of Docker
  - Public and private registries
    - Docker-hosted registries: Docker Hub (official one), Docker Store (for open-source and enterprise-verified images)
  - Open library of images

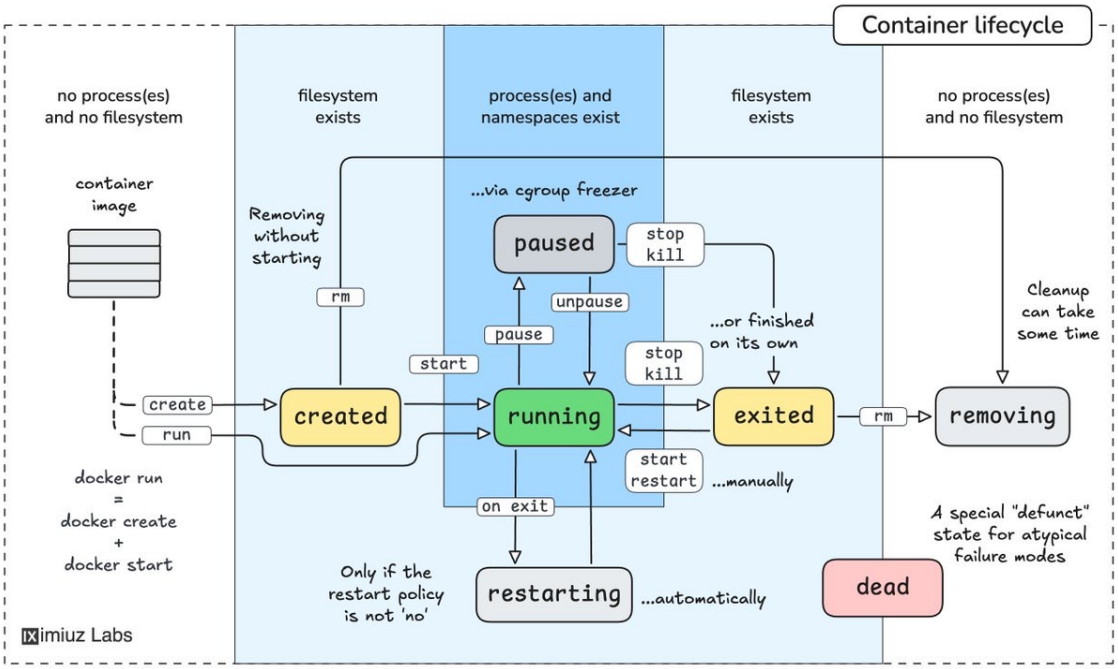
Docker: run container

- When you run a container from an image that is not yet installed locally but is available on Docker Hub, Docker will pull the image from the registry before starting the container



Docker container: states and transitions

- Different states a container can go through during its lifecycle and actions that trigger transitions



Docker commands: info

- Get system-wide info on Docker installation

  `$ docker info`

  including:
  - Number of images and containers, along with their status
  - Storage driver
  - Operating system, architecture, total memory
  - Docker registry information

Docker commands: image handling

- List all images on host

  `$ docker images`

  alternatively, `$ docker image ls`
- List all images, including intermediate image layers

  `$ docker image ls -a`
- You can filter and format the output to list images by name, tag, image digests (sha256) or image that meet specific conditions
  - E.g., to list untagged images (<none>) that are no longer associated with any tagged images (but consume disk space)

  `$ docker images --filter "dangling=true"`
- Remove an image

  `$ docker rmi imageid`

  alternatively, `$ docker image rm imageid`

can also use *imagename* instead of *imageid*

Docker commands: run

```
$ docker run [OPTIONS] IMAGE [COMMAND] [ARGS]
```

- Most common options
  - `--name` assigns a name to container
  - `-d` detached mode (runs container in background)
  - `-i` interactive (keeps STDIN open)
  - `-t` allocates a pseudo-tty, commonly used with `-i`
  - `--expose` exposes a port (or range of ports) inside container
  - `-p` publishes container's port (or range of ports) to host
  - `-v` mounts a volume
  - `-e` sets environment variables inside the container
  - `--link` adds link to another container

https://docs.docker.com/reference/cli/docker/container/run/

Docker commands: containers management

- List containers
  - Only running containers: `$ docker ps`
    alternatively, `$ docker container ls`
  - All containers (including stopped or killed containers):
    `$ docker ps -a`
- Manage container lifecycle
  - **Stop** a running container
    `$ docker stop containerid`
  - **Start** a stopped container
    `$ docker start containerid`
  - **Kill** a running container
    `$ docker kill containerid`
  - **Remove** a container (after stopping it)
    `$ docker rm containerid`

- Stop and remove a container

```
$ docker ps
$ docker stop containerid
$ docker ps -a
$ docker rm containerid
```

can also use *containername* instead of *containerid*

Docker commands: containers management

- Stop all containers

```
$ for i in $(docker ps -q); do docker stop $i; done
```

- Execute command in a running container

```
$ docker exec [OPTIONS] CONTAINER [COMMAND] [ARGS]
```

- Inspect a container
  - To get the most detailed view of a container environment

```
$ docker inspect containerid
```

- Copy files between host and container

```
$ docker cp containerid:path localpath
$ docker cp localpath containerid:path
```


Docker: networking

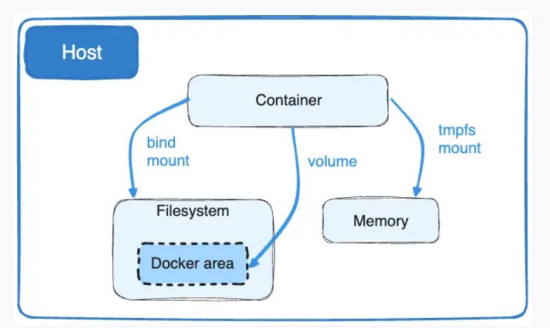
- Containers communicate with each other or with external systems
- Published ports
  - Use `-p hostPort:containerPort` (e.g., `-p 8080:80`) in `docker run` to expose a container port externally
  - Security issue: exposing container ports can be insecure
    - Restrict access to host only by binding to localhost (e.g., `-p 127.0.0.1:8080:80`)
  - Work only on host network: containers can communicate with each other via internal Docker network
- IP address and hostname
  - Containers get an IP address from Docker's dynamic subnet
  - Docker daemon manages subnetting and IP allocation
  - Default container hostname = container ID (can override with `--hostname`)

Docker: network drivers

- Docker networking is pluggable
  - Docker supports multiple network drivers that provide different networking behaviors, including:
- `bridge` (default)
  - Default network driver when no network is specified
  - Use when containers need to communicate on the same host
  - Implements a software bridge
    - Containers on the same bridge network can communicate with each other
    - Containers on different bridge networks are isolated
- `host`
  - Removes network isolation between container and host
  - Container uses host's network stack directly (no private IP assigned)

Docker: volumes

- Preferred mechanism for persisting data generated or used by containers
  - Volume content persists outside container lifecycle: use volumes instead of container writable layer for persistent data
  - Does not increase container image size
- How volumes work
  - Docker creates a directory inside its storage directory and manages its content
  - Default location on Linux: `/var/lib/docker/volumes/`
  - Volumes are created on demand if they do not already exist



Docker: volumes

- Mounting a volume: use `-v` or `--volume` in `docker run`

  `$ docker run … -v source:destination:[options]`

  – Add `ro` to mount the volume read-only
  – If the volume does not exist, Docker automatically creates it
- Managing volumes
  – Create: `$ docker volume create volumename`
  – List: `$ docker volume ls`
  – Inspect: `$ docker volume inspect volumename`
  – Remove: `$ docker volume rm volumename`
- Can also be declared in a Dockerfile using `VOLUME path`
- Working with volume data: pre-populate or load data using `docker cp`

https://docs.docker.com/engine/reference/commandline/

Docker volume: pros

✓ Fully managed by Docker
✓ Easy to back up or migrate
✓ Usable through Docker CLI or API
✓ Work on both Linux and Windows containers
✓ Shareable across multiple containers
✓ Can store encrypted content
✓ Suitable for pre-populated or write-heavy workloads (e.g., databases)

Docker hands-on

- Download and install Docker
  - Available on multiple platforms

  https://docs.docker.com/get-started
- Test Docker installation

  ```
  $ docker --version
  ```
- Run the default hello-world container

  ```
  $ docker run hello-world
  ```
- Run a “Hello World” message using Alpine Linux

  ```
  $ docker run alpine /bin/echo 'Hello world'
  ```
  - alpine: lightweight Linux distro with very small image size
- Use commands to:
  - List containers and container images
  - Stop and remove containers, remove container images

Docker hands-on: networking

- Run nginx Web server inside a container
  - Bind container port 80 to host port 80

  ```
  $ docker run -dp 80:80 --name web nginx
  ```

  Flag `-p`: publish container port (80) to host port (80)

  Flag `-d`: run in detached mode

1. Send HTTP request through Web browser
   - First retrieve hostname of host machine (e.g., localhost)
2. Send HTTP requests to nginx from another interactive container using a custom bridge network

```
$ docker network create -d bridge my_net
$ docker run -dp 80:80 --name web --network=my_net nginx
$ docker run -it --network=my_net --name web_test busybox
/ # wget -O - http://web:80/
/ # exit
```

Docker hands-on: from Dockerfile

- Running Apache web server with minimal index page
  1. Define container image with Dockerfile
     - Define image starting from Ubuntu, install and configure Apache
     - Incoming port set to 80 using EXPOSE instruction

```
FROM ubuntu:18.04

# Install dependencies
RUN apt-get update -y && \
    apt-get install -y apache2 && \
    apt-get clean

# Add simple web page
RUN echo "Hello World!" > /var/www/html/index.html

# Expose port and run Apache in the foreground
EXPOSE 80
CMD ["apache2ctl", "-D", "FOREGROUND"]
```


Docker hands-on: from Dockerfile

2. Build container image from Dockerfile

```
$ docker build -t hello-apache .
```

3. Run container and bind

```
$ docker run -dp 127.0.0.1:8080:80 hello-apache
```

4. Execute an interactive shell in running container

```
$ docker exec -it hello-apache /bin/bash
```

- To reduce container image size, avoid creating unnecessary layers
  - E.g., in Dockerfile combine `apt-get update` and the installation of multiple packages into a single RUN instruction (see slide 46)
  - Use `\` to split long commands across multiple lines

Docker hands-on: volumes

- Run nginx container with volume

```
$ docker volume create my-vol
$ docker volume ls
$ docker volume inspect my-vol
$ docker run -d \
    --name devtest \
    -v my-vol:/app \
    nginx:latest
```

  - `my-vol` is the source, `/app` is the target inside container

```
$ docker inspect devtest
```

  - Inspect the container to verify that Docker has created the volume and mounted it correctly

Docker: optimize Docker images

- Fewer layers → smaller images → faster builds and deployments
- Why optimize Docker images?
  - Essential for DevOps engineers at every stage of CI/CD process
  - Reduces image size and disk usage
  - Speeds up image transfer, deployment, and startup times
  - Improves security by reducing the attack surface
  - Best practice used by Google and other major tech companies
  - Best practice employed by Google and other tech giants

https://devopscube.com/reduce-docker-image-size

Docker: optimize Docker images

- Techniques
  1. Use minimal base images (e.g., alpine, minideb) or distroless base images
     - Distroless images:
       - contain only application and its runtime dependencies
       - include package managers, shells, or other common utilities
       - More secure, smaller, harder to tamper with https://github.com/GoogleContainerTools/distroless
  2. Minimize the number of image layers
     - Combine related commands in a single RUN instruction
     - Avoid unnecessary COPY, ADD, or repeated RUN steps

Docker: optimize Docker images

- Techniques
  3. Multistage builds
     - Use intermediate images (build stages) to
       - compile code
       - install dependencies, and package files
     - Final image contains only the files and libraries needed to run app

```
# Build stage
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

# Final stage
FROM alpine:3.20
COPY --from=builder /app/myapp /usr/local/bin/myapp
CMD ["myapp"]
```

Docker: optimize Docker images

- Techniques
  4. Exploit layer caching
     - Place instructions that change infrequently (like installing dependencies) before COPY commands
     - Docker reuses cached layers for faster builds when source code changes.
  5. Use a `.dockerignore` file
     - Specify files and directories to exclude from the build context
     - Common exclusions: `node_modules`, `.git`, `*.log`, `__pycache__/`
  6. Keep application data in a volume
     - Avoid storing persistent data inside the container.
     - Use Docker volumes to store databases, logs, uploaded files

Resizing containers

- Just like VMs, containers can be resized and migrated
- Resizing allows changing CPU, memory, and I/O limits dynamically
  - Note: on Docker, dynamic resizing is not supported on Windows

```
$ docker update [OPTIONS] CONTAINER [CONTAINER...]
```

  - Examples

```
$ docker update --cpu-shares 512 containerID
$ docker update --cpu-shares 512 -m 300M containerID
```

Docker: sizing containers

- By default, a container has no resource constraints
  - Can use as much resource as host's kernel scheduler allows
- Docker provides ways to control how much memory or CPU a container can use by setting runtime configuration flags of `docker run`
  https://docs.docker.com/engine/containers/resource_constraints/
  - Docker Engine implements configuration changes by modifying settings of container's `cgroup`

Docker: sizing containers - memory

- Avoid running out of memory (OOM)
  - Individual containers can be killed
  - Docker daemon has lower OOM score, so less risk than containers
- Docker can enforce hard or soft memory limits
  - Hard limit: container cannot use more than a given amount of user or system memory; `--memory` flag
  - Soft limit: container can use as much memory as it needs unless certain conditions are met, such as when kernel detects contention or low memory on host machine
  - Example: limit container to use at most 500 MB of memory (hard limit) and specify also a soft limit

```
$ docker run –it --memory-reservation="300m" \
  --memory="500m" ubuntu /bin/bash
```

Docker: sizing containers - CPU

- Various constraints to limit container usage of host machine's CPU cycles
- Some options

  `--cpus=<value>`: limit how many CPU resources a container can use (hard limit)

  `--cpu-quota=<value>`: set CPU Completely Fair Scheduler (CFS) quota on container

  `--cpuset-cpus`: limit specific CPUs or cores a container can use

  `--cpu-shares`: set to value >/< 1024 to increase/reduce container's weight, and give it access to greater/less proportion of CPU cycles (soft limit)

  – Example: limit container to use at most 50% of CPU every second

  ```
  $ docker run -it --cpus=".5" ubuntu /bin/bash
  ```

  Alternatively,

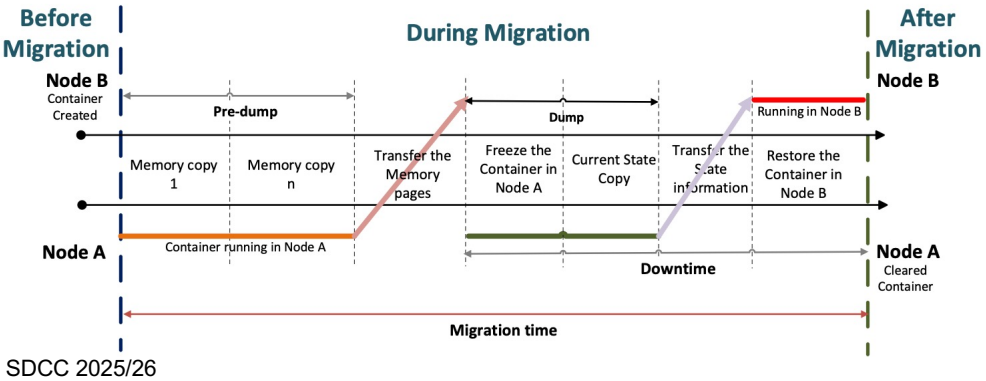
  ```
  $ docker run -it --cpu-period=100000 \
  --cpu-quota=50000 ubuntu /bin/bash
  ```


Live migration of containers

- As for VM migration, we need to:
  – Save state
  – Transfer state
  – Restore from state
- State saving, transferring and restoring happen with frozen app: migration downtime
  – Use memory pre-copy or memory post-copy
- No native support in container engines, requires additional tool
- We also need to migrate container image (and volumes) and network connections

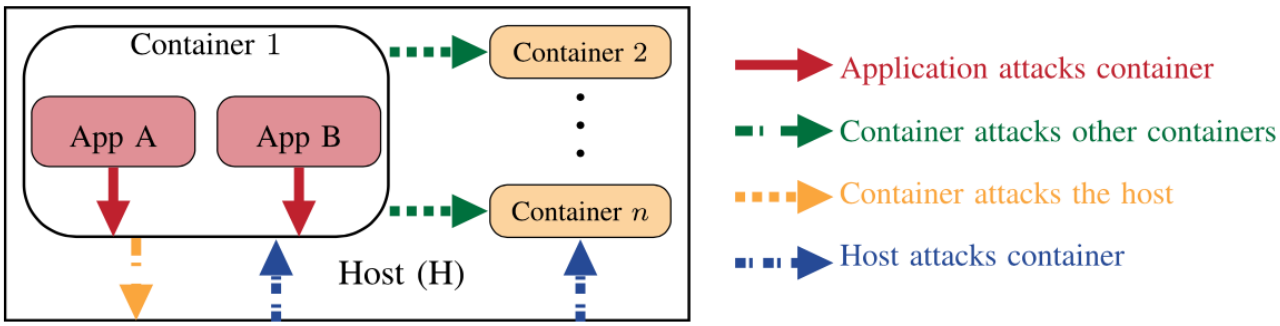
Live migration of containers

- Use CRIU tool to support live migration (in Docker and other container engines) through checkpointing and restoration https://criu.org/
  - During checkpoint, CRIU freezes running container at source host and collects information about its CPU state, memory content, and process tree
  - Collected information is passed on to destination host, and container is resumed
  - How to: https://docs.docker.com/reference/cli/docker/checkpoint/



Container security

- Where attacks come from in a containerized environment?



- Example of attack: *container escape and privilege escalation*
  - Attacker can leverage containerized app's vulnerabilities to breach its isolation boundary, gaining access to host system's resources
  - Once attacker accesses host, it can escalate its privilege to access other containers or run harmful code on host